Second piece: Concurrency

↳ multiple activities that can happen at the same time.

What are some real-world example of concurrency?

In an orchestra, different instruments progress together at overlapping times.

outside computers

On roads, one vehicle drives north and other drives south

In bands, they all overlap in time, creating a co-ordinated sound.

concurrency challenge for OS developers

- manage hardware resources
- provide responsiveness to users
- run multiple applications simultaneously.

concurrency is also a concern for many application developers.

eg:. Network services like Google, Amazon, etc. need to be able to handle multiple requests from their clients.

How can you write a program with dozens of events happening at once? write a concurrent programs.

one with simultaneous activities → using threads, a set of sequential streams of execution.

i.e., we can represent each concurrent task as a thread.

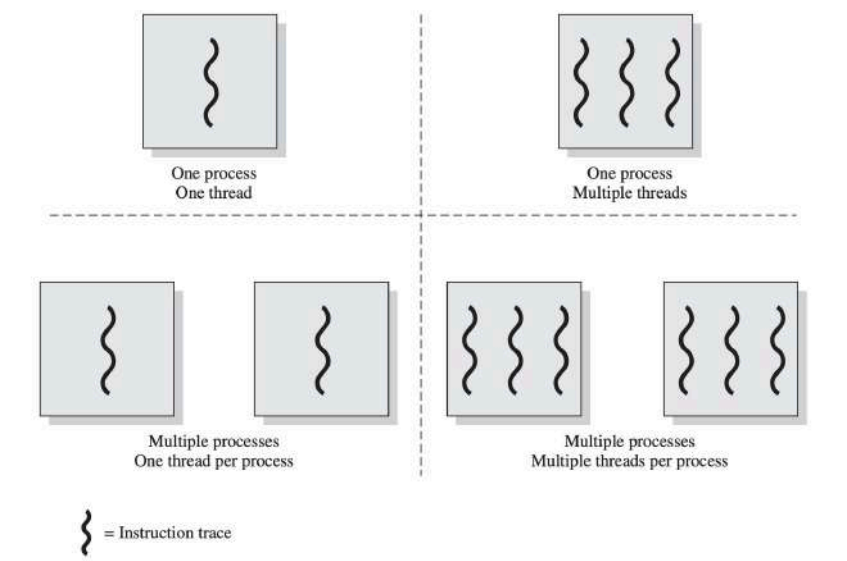
an abstraction of a sequential execution.

Traditional program → single threaded (one logical sequence of steps)

Multi-threaded program → a generalization of the basic programming model

each individual thread follows a single sequence of steps.

eg:- MS-DOS



eg:- Java run-time environment

eg:- some variants of unix

eg:- windows, solaris, modern versions of unix

An example of Multi-threading.

Consider an earth visualizer application like, Google Earth.

- you can fly anywhere in the world virtually
- you can zoom in/out, pan, tilt, rotate the view
- the application shows aerial/satellite images at multiple resolutions (blurry at first, sharper as more data loads)
- it can overlay extra information like roads, names of cities, weather, etc.

Key requirement: the user's control must always be responsive.

even if the program is fetching large amounts of data (like a sharper image of an area), the user should still be able to move around, click, and search without the app freezing.

Suppose the program is written sequentially, the tasks are:

1. Draw screen portion (render the visible map tiles) ?see
2. Display UI widgets (buttons, menus, search bar)
3. Processor user input (mouse moves, clicks, keyboard)
4. Fetch higher resolution images (from local cache/internet)

In a sequential step, these would happen one after another.

- Imagine the program is fetching a 50 MB high-resolution satellite image → during this time, the program cannot accept your mouse movement or key press.

Then app feels "frozen" until the fetch finishes.

↳ bad for interactive applications.

Multithreaded (concurrent approach)

with threads, the programmer can split the responsibilities.

- UI thread → always running, processing user input (mouse, keyboard, search box, etc.)
- rendering thread → continuously re-draws the map view as the camera changes.
- Image fetching thread (s) → runs in the background to download and load higher resolution images.
- Data overlay thread (s) → fetches extra info (labels, roads, points of interest)

Now, if the user drags the mouse to pan, the UI thread instantly reacts.

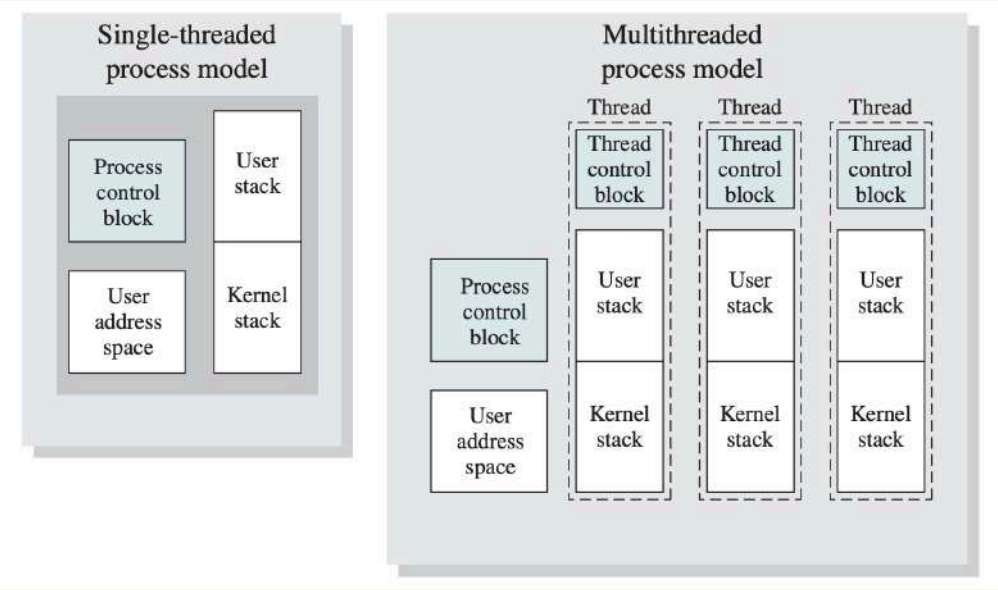
The rendering thread shows a blurred/low-resolution version immediately, so feedback is instant. Meanwhile, the fetching thread downloads sharper images. As soon as they arrive, the rendering thread updates the view smoothly.

The user never feels blocked. The app is interactive and responsive.

Threads allow multiple execution flows within the same process.

Question 1: what must be shared so that threads can truly work together?

Question 2: what must be private so that they don't corrupt each other's execution?

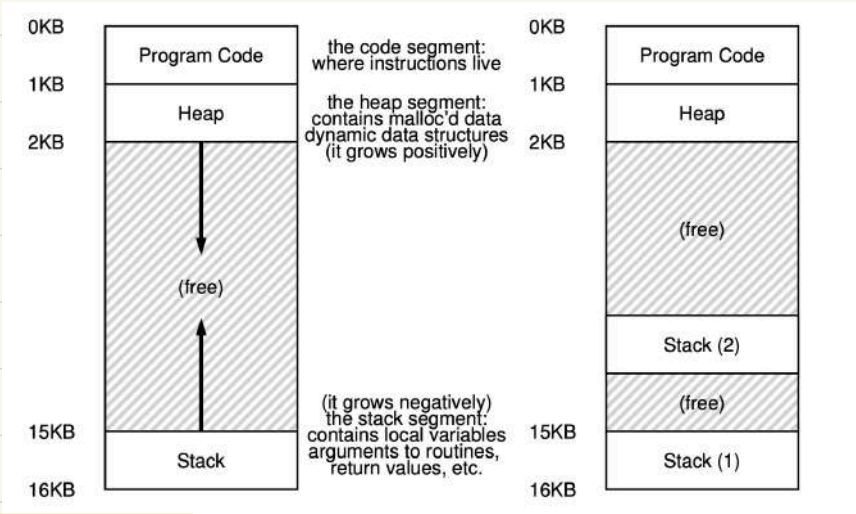


Why threads share certain things?

1. Address space (code, globals, heap)

Goal of threads: multiple workers acting on the same data.

With different address space, communication and sharing results wouldn't be efficient.



2. Open files
of threads didn't share this, one thread couldn't log to the same file another thread opened.

3. Process-wide attributes (PID, working directory, etc.)
these belongs to the process as a whole, not to individual threads.

Why threads not share certain things

1. Program counter (PC)
- each thread must be able to execute different instructions independently.
- without a separate PC per thread, the OS couldn't resume thread correctly after a context switch.

2. Registers
- registers hold temporary state (eg:- results of calculations, loop counters)
- shared register → one thread's computation could overwrite another's.

3. **stack (user + kernel)**

- the stack stores local variables and function calls.
- shared stack → * thread A's function call could overwrite Thread B's local variables.
  * return address would clash, crashing the program
- private stacks → ensures each thread's execution path is independent.
  kernel stacks are also private → the OS must handle system calls from different threads without mixing them up.

4. **Thread ID:**

needed to distinguish threads within the same process.
O/w, the OS and program couldn't identify which thread did what.

In short, threads share resources (memory, files, environment) because that's the whole point → easy communication and co-operation

Threads keep private execution state (PC, registers, stacks, IDs) because without it they couldn't run independently or safely.

Advantages of using threads

* Program structure : expressing logically concurrent tasks.

* Responsiveness : shifting work to run in the background.

  - It is common to create threads to perform work in the background, without the user waiting for the result → user interface remain responsive.

    For eg: In a web browser, the cancel button should continue to work even (or especially!) if the downloaded page is gigantic or a script on the page takes a long time to execute.

- OS kernels make extensive use of threads to preserve responsiveness

  For eg: When writing a file, OS stores the modified data in a kernel buffer, and returns immediately to the application. In the background OS kernel runs a separate thread to flush the modified data out of the disk.

  Similarly, on file reads, the kernel can have a thread which attempts to anticipate which blocks are likely to read next and bring those blocks from disk before the application asks for them.

* Performance : exploiting multiple processors.
  programs can use threads on a multi-processor to do work in parallel → same work in less time.

* Performance : managing I/O devices
  By running tasks as separate threads when one task is waiting for I/O, the processor can make progress on a different task.